A lookup table has a different usage pattern than a stack or a queue. Whereas almost every operation on a stack or a queue modifies it in some way, either to add an element or remove one, a lookup table might be modified rarely. The simple DNS cache in listing 3.13 is one example of such a scenario, which features a greatly reduced interface compared to `std::map<>`. As you saw with the stack and queue, the interfaces of the standard containers aren't suitable when the data structure is to be accessed from multiple threads concurrently, because there are inherent race conditions in the interface design, so they need to be cut down and revised.

The biggest problem with the `std::map<>` interface from a concurrency perspective is the iterators. Although it's possible to have an iterator that provides safe access into a container even when other threads can access (and modify) the container, this is a tricky proposition. Correctly handling iterators requires you to deal with issues such as another thread deleting the element that the iterator is referring to, which can get rather involved. For the first cut at a thread-safe lookup table interface, you'll skip the iterators. Given that the interface to `std::map<>` (and the other associative containers in the standard library) is so heavily iterator-based, it's probably worth setting them aside and designing the interface from the ground up.

There are only a few basic operations on a lookup table:

- Add a new key/value pair.
- Change the value associated with a given key.
- Remove a key and its associated value.
- Obtain the value associated with a given key if any.

There are also a few container-wide operations that might be useful, such as a check on whether the container is empty, a snapshot of the complete list of keys, or a snapshot of the complete set of key/value pairs.

If you stick to the simple thread-safety guidelines such as not returning references and put a simple mutex lock around the entirety of each member function, all of these are safe; they either come before some modification from another thread or come after it. The biggest potential for a race condition is when a new key/value pair is being added; if two threads add a